

Zeequent Batch Auctions: An Overview

Kimia Esmaili¹, Elizabeth van Oorschot², and Jeremy Clark¹

¹ Concordia University, Montreal, Canada
kimia.esmaili@mail.concordia.ca
j.clark@concordia.ca

² McGill University, Montreal, Canada

Abstract. Frequent batch auctions (FBAs) are an alternative market structure to continuous limit order books (CLOBs), replacing immediate trade execution with periodic clearing over short time intervals. By clearing orders together at a uniform price, FBAs reduce the value of sub-second latency advantages, which are exploited by high-frequency traders (HFTs). This paper gives an overview of a zero-knowledge approach to batch auctions, in which the operator commits to the order batch and proves with a custom zk-SNARK that the published clearing price and allocation satisfy the auction rule without revealing the underlying orders.

Keywords: frequent batch auctions, call markets, zero-knowledge proofs

1 Introductory Remarks

Modern stock exchanges (*e.g.*, NASDAQ and NYSE) use two main trading systems [6]. During regular trading hours, traders can submit orders to a continuous limit order book (CLOB), where compatible buy and sell orders can execute as soon as they meet. Traders can also submit orders to a scheduled auction (or call market), where orders are held until a fixed time—the end of the trading day—and then executed together. Near the end of the day, this auction becomes the exclusive way to trade and then at market close, it clears as many trades as possible at a single price, which becomes the closing price of the stock. The same auction mechanism is used before the market opens the next day to establish an opening price.

Closing auctions play an important role because the closing price is widely used as a reference point: traders benchmark their portfolios against it, indexes are calculated from it, and derivatives settle using it. By aggregating many orders and clearing them together at a single price, the closing auction also makes the official closing price less dependent on any single last-minute trade, which make it robust against simple forms of manipulation.

Across a typical trading day, most market activity occurs in the CLOB. Because CLOB orders are processed as they arrive, small differences in speed can determine who trades first. High-frequency traders (HFTs) use fast computers, private data links, and automated strategies to exploit these differences, sometimes investing heavily to shave milliseconds from communication times between

markets. This has led to concerns that continuous markets reward speed too heavily. The counterpoint is that HFTs can also narrow spreads, add liquidity, and help prices adjust quickly to new information. But even if HFTs narrow spreads, the resulting liquidity may not be beneficial when it can be withdrawn faster than other traders can respond.

1.1 Frequent Batch Auctions (FBAs)

In an opening or closing auction, orders are collected over time and executed together at a single price. Since trades are not matched one-by-one as orders arrive, reaching the exchange faster by a few microseconds provides little direct advantage. This observation motivates frequent batch auctions (FBAs), proposed by Budish, Cramton, and Shim [2]. An FBA takes the basic idea of an opening or closing auction and applies it repeatedly throughout the trading day. Instead of using a CLOB, an exchange can collect orders over short intervals (*e.g.*, each second) and then clear all the trades within that second using an auction. Since arriving at the start of the batch is no different from arriving the end of the batch (there is no time priority, only price priority), traders can stop competing over speed and focus their strategies on price.

1.2 Transparency

FBAs come with at least one disadvantage compared to CLOBs: transparency. In a CLOB, the order book is visible. A trader who submits an order may not know what other orders will arrive first, or how the book will change during the delay between submission and execution, but once the order reaches execution time, there is no uncertainty about why it executed, did not execute, or executed at a particular price.

Batch auctions are different. Orders are collected over a time window and cleared only after the window closes. During that interval, no participant can see the full set of orders that will determine the clearing price, and after the auction closes, participants typically see only the final outcome, not all the trades that failed to execute (even though the closing price of the auction depended on these invisible trades). This makes the auction less transparent: a trader can observe whether their own order executed and at what price, but cannot independently verify that the exchange included all eligible orders, excluded ineligible ones, chose the correct clearing price, and allocated trades appropriately.

1.3 Zero-Knowledge Proofs

Cryptography offers a bridge between correctness and secrecy. Any computation can, in principle, be accompanied by a proof that it was performed correctly; with a zero-knowledge proof (ZKP) [5], this can be done without revealing the private inputs to the computation. Applied to batch auctions, this means an exchange could prove that the published clearing price and allocation follow from the submitted orders, without revealing the orders themselves.

In this research, we develop a zero-knowledge frequent batch auction (zk-FBA) protocol: a batch auction in which the exchange publishes the auction outcome together with a cryptographic proof that the clearing price and allocation were computed correctly. We use zk-SNARKs [4], which are modern zero-knowledge proofs designed to be compact and fast to verify. In designing the protocol, we do not rely on general-purpose tools for developing zk-SNARKs which are computationally expensive for the exchange which needs to produce a proof at the end of each batch auction (*e.g.*, once a second). Instead we design a leaner, custom mathematical representation of the clearing rule; in cryptographic terms, this is a custom arithmetization [8] for a polynomial interactive oracle proof (PIOP) [3].

2 zk-FBAs

We now turn to our protocol. In this short paper, we provide only an overview of the approach. We refer the interested reader to our GitHub repository for the concrete cryptographic details.³ Frequent batch auctions (FBAs) are primarily an academic market-design proposal [1] rather than a widely deployed, operational standard, so there is no single canonical specification of exactly how it should operate. In this research, we fill on some concrete details by adapting the structure of a modern closing auction (in the style of NASDAQ’s Closing Cross), just with the assumption that it runs frequently (*e.g.*, once a minute instead of once a day).

2.1 Order Types

Like a CLOB or closing cross, each single asset (*e.g.*, an equity like APPL, token like ETH, a contract like June 2026 WTI crude oil futures) will have a dedicated FBA. Our zk-FBA supports three order types: limit, market, and imbalance-only (IO).⁴ Each can be a buy or sell order.

A *limit* order specifies a limit price ℓ . Let p^* denote the market’s closing (clearing) price for the batch. A buy limit order (bid) is eligible to execute only if $p^* \leq \ell$, while a sell limit order (ask/offer) is eligible only if $p^* \geq \ell$.

A *market* order specifies only a quantity (with no limit price). A buy market order is willing to execute at any closing price p^* , and a sell market order is likewise willing to execute at any closing price p^* ; thus market orders are always eligible at p^* .

An *imbalance-only* (IO) order is a special type of order that resembles a limit order: it is either a buy order or a sell order with limit price ℓ . However, IO orders are eligible only if they reduce the residual imbalance at the closing price p^* . Concretely, a buy IO is eligible only if $p^* \leq \ell$ and there is excess supply at p^* (*i.e.*, more volume offered for sale than there are buyers). In that case, the buy

³ GitHub: <https://github.com/MadibaGroup/2024-zkFBA>

⁴ These correspond to LOC, MOC, and IO orders in NASDAQ’s closing cross.

IO may execute to reduce the imbalance. A sell IO is the converse: it is eligible only if $p^* \geq \ell$ and there is excess demand at p^* . The purpose of IO orders will be clearer after discussing how a market closes.

2.2 Market Closing

At the end of each batch, the exchange computes a closing price p^* in three conceptual phases. *Phase 1*: Using only limit and market orders (*i.e.*, excluding IO), the exchange considers each price tick p and computes the cumulative buy and sell depth at p , denoted $D(p)$ and $S(p)$. The executable volume at p is $V(p) = \min(D(p), S(p))$. Let $V_{\max} = \max_p V(p)$ which is the price tick(s) that will allow the greatest volume of trades to execute. If $V_{\max}$ occurs at a single price, we set p^* to that price and move to phase 3 directly. If $V_{\max}$ occurs at a contiguous⁵ interval of prices $[c, d] = \{p : V(p) = V_{\max}\}$, we move to phase 2.

If we enter *Phase 2*, we have a set of prices that maximize trading volume, so the tie-breaker is to minimize the imbalance in supply and demand. For each $p \in [c, d]$ we compute the trade imbalance $\Delta(p) = D(p) - S(p)$, and select p^* by minimizing the imbalance magnitude $|\Delta(p)|$ over $p \in [c, d]$ (with a deterministic tertiary rule if still tied). This gives a final market clearing price p^* .

Phase 3: Given p^* , the exchange executes all eligible limit and market orders at p^* . Orders strictly better than p^* execute in full; such limit orders receive price improvement (a buy executes at $p^* \leq \ell$ and a sell executes at $p^* \geq \ell$). Limit orders priced exactly at p^* are eligible to execute, but if eligible buy and sell volume at p^* are unequal then one side is short and the other side is long. The exchange then admits eligible IO orders on the short side at p^* to reduce the residual imbalance. If an imbalance remains, the remaining at-price interest on the long side is rationed pro-rata within the batch.

2.3 Zero-Knowledge

The zk-FBA protocol works as follows. The exchange creates arrays indexed by price ticks $p \in \mathcal{P}$: one array for limit-order bid volume $\text{BidsVol}[p]$, one for limit-order ask volume $\text{AsksVol}[p]$, and analogous arrays for IO volume $\text{IOBidsVol}[p]$ and $\text{IOAsksVol}[p]$. At the start of a batch these arrays are all zero. The exchange commits to each array by interpolating it as a polynomial over the tick domain and committing with a polynomial commitment scheme [7] and the resulting commitments are published (publicly).

When a trader submits an order, the exchange updates the appropriate array entry with the added volume. For limit/IO orders, this is at only the tick corresponding to its limit price. For market-order quantities, they are encoded by adding to the highest tick for buys and the lowest ticks for sells. The exchange commits and publishes the new auction book and gives the trader a short *update*

⁵ Cumulative buy depth $D(p)$ is non-increasing while cumulative sell depth $S(p)$ is non-decreasing, so $V(p)$ can only rise (weakly) up to the crossing region and then fall (weakly).

receipt: an opening proof showing that the committed polynomial changed by only the correct amount at only the correct tick.

Upon close of a batch, the exchange runs Phases 1–3 above and produces a public *clearing receipt*. Concretely, it materializes (and commits to) tick-indexed arrays for: (i) limit-order histograms $\text{BidsVol}[p]$ and $\text{AsksVol}[p]$, (ii) IO histograms $\text{IOBidsVol}[p]$ and $\text{IOAsksVol}[p]$, (iii) cumulative depth curves $D(p)$ and $S(p)$, (iv) executed volume $V(p)$, and (v) the leftover/surplus variables on each side at each tick (used to express imbalance and pro-rata allocation at p^*).

The proof system enforces the market logic by stating algebraic relationships among these committed arrays. At a high level, the computation uses only a small set of gadgets: addition, subtraction, minimum, maximum, absolute value, non-negative check, and one-hot selectors. Several of these are common in the style of proof we use, while the others we were able to build from the standard gadgets.

3 Concluding Remarks

A practical limitation of zk-FBAs is that despite using very efficient proof techniques, it is not necessarily cheap enough. Producing a cryptographic certificate for every market after every minute (or faster) may simply be too slow or too costly, even with modern tooling. In future work, we will implement and optimize our protocol. The benchmark it, we will also generate synthetic datasets for what order flow might look like for a real market (*e.g.*, APPL) adapted to a FBA. The takeaway is that we can choose how often to run the auction and how much to reveal, and justify those choices with benchmarks rather than intuition.

References

1. Budish, E., Cramton, P., Shim, J.: Implementation details for frequent batch auctions: Slowing down markets to the blink of an eye. *American Economic Review* **104**(5) (2014)
2. Budish, E., Cramton, P., Shim, J.: The high-frequency trading arms race: Frequent batch auctions as a market design response. *The Quarterly Journal of Economics* **130**(4), 1547–1621 (2015)
3. Bünz, B., Fisch, B., Szepieniec, A.: Transparent snarks from dark compilers. In: *EUROCRYPT* (2020)
4. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive* (2019)
5. Goldreich, O., Micali, S., Wigderson, A.: How to prove all np statements in zero-knowledge and a methodology of cryptographic protocol design. In: *CRYPTO* (1986)
6. Harris, L.: *Trading and exchanges: market microstructure for practitioners*. Oxford (2003)
7. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: *ASIACRYPT*. pp. 177–194. Springer (2010)
8. van Oorschot, E., Deng, Y., Clark, J.: *Plonkbook: A handbook for poly-iop gadgets*. GitHub Pages (2024), <https://www.plonkbook.org>